



IBT–CodeOps Full-Stack Software Development Program

Module 3 — React Project Brief ·

ADDIS EATS

A React Frontend for a Food-Ordering App in Addis Ababa

Project Overview

The brief is fixed so everyone is assessed on the same ground. What varies from student to student is how well the application is structured, not what it is asked to do. Invention and your own product idea belong in the capstone project, not here. the goal is to assemble everything you have written since Day 26 into one finished.

This document also lists the full customer and admin feature sets from the original Addis Eats specification (see Customer-Facing Features and Admin Features below). The customer list is the scope of the required build; the admin list is an optional extension for students who want to push further — it is built the same way, as a second, guarded route group inside the same React app, not a separate project.

Screens & Routes

Your application must expose exactly the five customer routes below, wired up with React Router.

Screen	Route	What it does
Home	/	Today's specials, with a link into the menu
Menu	/menu	Dishes fetched from the API; category filter reflected in the URL
Dish	/menu/:id	Full detail for one dish, with an add-to-cart action
Cart	/cart	Order lines and the running ETB total
Checkout	/checkout	Validated order form, guarded behind sign-in

Additional Customer Routes

Screen	Route	What it does
Favorites	/favorites	Saved dishes; toggled from a heart icon on any DishCard (feature 17)
Order History	/orders	Past orders, each with a one-click Reorder action (features 16, 19)

Admin Routes

If you build the admin extension, add these as a nested, guarded route group under `/admin` — a second, natural place to practise Day 31's nested, dynamic and guarded routes.

Screen	Route	What it does
Admin Login	<code>/admin/login</code>	Username / password gate for the admin area
Admin Dashboard	<code>/admin</code>	Nested layout: revenue, order count, top dishes, status breakdown
Admin Menu	<code>/admin/menu</code>	CRUD list for dishes — search, add, edit, delete
Admin Orders	<code>/admin/orders</code>	List every order; view details; update status; delete

Customer-Facing Features (26 total)

These are the same 26 customer features from the original Addis Eats specification, built here as React components and screens instead of static HTML/CSS/JS pages.

Core Features —

Every one of these must be completed and mapped onto the five screens above.

#	Feature	What it means
1	Browse Menu	Load and display all dishes, fetched from the API
2	Live Search	Filter the menu instantly as the customer types
3	Category Filter	Filter by Ethiopian, Pizza, Burgers, Drinks
4	Add to Cart	Add a dish to the shopping cart
5	Update Quantity	Increase or decrease item quantity with +/- controls
6	Remove from Cart	Remove a single item from the cart
7	Live ETB Total	Cart total recalculates instantly as items change
8	Cart Persistence	Cart survives a page refresh (context/store + localStorage)
9	Checkout Form	Collect customer name, phone, and delivery area
10	Form Validation	Validate name, phone number format, and area before submit
11	Order Confirmation	Show a confirmation message/screen after an order is placed
12	Loading States	Show a loading indicator while data is being fetched
13	Empty States	Show a friendly message when the cart or menu is empty
14	Error Handling	Handle and display errors gracefully (e.g. failed data load)
15	Responsive Design	Mobile-first layout that works on phone, tablet, and desktop

Additional Features

Optional. Complete as many as time allows, once the 15 core features and the Day 26–34 requirements are solid.

#	Feature	What it means
16	Order History	Let the customer view their past orders
17	Favorites / Wishlist	Let the customer save favorite dishes
18	Dish Detail View	Show full details (ingredients, description) for one dish
19	Reorder Functionality	Reorder a past order with one click
20	Delivery Fee Calculator	Calculate delivery fee, e.g. based on area
21	Estimated Delivery Time	Show an estimated delivery time at checkout
22	Dark / Light Theme Toggle	Let the customer switch and persist a theme
23	Special Instructions	Let the customer add a note to their order
24	Cart Badge Count	Show the number of items in the cart icon/badge
25	Keyboard Accessibility	Full site usable via keyboard navigation
26	Formatted Currency (ETB)	Prices consistently formatted as Ethiopian Birr

Admin Features (17 total)

Building the admin side turns Addis Eats into the two-part application from the original specification, built entirely in React behind the /admin route group above.

Authentication & Dashboard (5)

#	Feature	What it means
1	Admin Login	Username/password login for the admin
2	Session Management	Keep the admin logged in via context + sessionStorage
3	Dashboard Analytics	Show revenue, order count, and average order value
4	Top Selling Dishes	List the best-selling dishes
5	Order Status Distribution	Visualize how many orders are in each status

Menu Management — CRUD (5)

#	Feature	What it means
6	View All Dishes	List all dishes with a search box
7	Add New Dish	Create a new dish entry
8	Edit Dish	Update an existing dish's details
9	Delete Dish	Delete a dish, with a confirmation step
10	Search Dishes	Filter the admin dish list by keyword

Order Management (4)

#	Feature	What it means
11	View All Orders	List every customer order
12	Update Order Status	Move an order through pending → preparing → delivering → delivered
13	Delete Order	Remove an order from the system
14	Order Details View	See customer info, items, and total for one order

Data & System (3)

#	Feature	What it means
15	Data Persistence	All admin data (dishes, orders) saved to localStorage
16	Data Seeding	Load initial data from menu-data.json on first run
17	Logout	End the admin session and return to /admin/login

Extension, not a substitute

The Day 26–34 requirements table below still applies to the core customer build. The admin features add breadth (more CRUD, more routes, more state) — they do not replace any required customer feature or screen.

Plan Before You Code

Where State Lives (you can customize the following state and where it belongs)

Ask who needs to read each piece of state and who needs to change it, then place it in the lowest component that contains all of them. Nothing should be global by default. Take the following as initiation but not mandatory.

State	Where it belongs
Selected category	The URL — shareable, survives refresh
Fetches dishes	The component that displays them (Menu)
The order / cart	A store — read by Menu, Dish, Cart and Checkout
Sign-in session	Context — rarely changes, needed by the route guard
Checkout form fields, incl. special instructions	The Checkout component, and nowhere else
Whether a modal is open	The component that opens it
Favorite dish IDs	A store, persisted to localStorage — read by DishCard hearts and Favorites
Order history	A store, appended after a successful checkout — read by OrderHistory
Theme preference	Context, persisted to localStorage — read by Layout, set from anywhere
Delivery fee / estimated time	Derived in Cart or Checkout from the selected area — not stored on its own
Cart badge count	Derived from the cart store — no state of its own

Recommended Folder Structure

Group files by feature, not by file type. A components/ folder with thirty files tells you nothing; a feature folder can be understood — and deleted — on its own. The following structure is initiation you can customize as you want but like the following group files by feature.

```

addis-eats/
├── public/
│   ├── menu-data.json  # seed data the app fetches, standing in for an API
│   ├── src/
│   ├── api/           # fetch helpers, one per resource
│   │   └── dishes.js
│   ├── hooks/         # reusable hooks only — useFetch, useDebounce
│   ├── ui/            # generic UI only — Button, Modal, Spinner
│   ├── utils/         # formatCurrency.js (26), deliveryEstimate.js (20–21)
│   ├── menu/          # Menu, CategoryBar, DishList, DishCard
│   ├── cart/          # cart store, CartPanel, CartBadge (24)
│   ├── checkout/      # Checkout, validate.js, Field, DeliveryEstimate
│   ├── favorites/     # (booster) saved dishes + toggle — feature 17
│   │   ├── Favorites.jsx, FavoriteButton.jsx, favoritesStore.js
│   ├── orders/        # (booster) order history + reorder — features 16, 19
│   │   ├── OrderHistory.jsx, OrderHistoryItem.jsx, orderHistoryStore.js
│   ├── theme/         # (booster) ThemeContext, ThemeToggle — feature 22
│   │   ├── ThemeContext.jsx, ThemeToggle.jsx
│   ├── auth/          # AuthProvider, useAuth, RequireAuth (customer sign-in)
│   ├── admin/         # (admin extension — optional)
│   │   ├── AdminLogin.jsx  # feature 1: Admin Login
│   │   ├── useAdminAuth.js # feature 2: Session Management
│   │   ├── RequireAdmin.jsx # guard for the whole /admin group
│   │   ├── AdminLayout.jsx # nested layout: sidebar + <Outlet />
│   │   ├── Dashboard.jsx  # features 3–5: analytics, top dishes, status chart
│   │   ├── DishManager.jsx # features 6–10: view, add, edit, delete, search
│   │   ├── DishForm.jsx   # add/edit form used by DishManager
│   │   └── OrderManager.jsx # features 11–14: view, status, delete, details
│   ├── App.jsx
│   ├── Layout.jsx
│   └── main.jsx
├── .gitignore
├── package.json
└── README.md
  
```

ui/ is the exception

Genuinely generic components — a button, a spinner, a modal shell — belong together in one shared folder. If a component mentions a dish or an order, it is not generic and belongs in its feature folder

Keyboard accessibility isn't a folder

Feature 25 is a cross-cutting practice, not a screen or a store — it means every interactive element in every folder above (buttons, cards, form fields, the theme toggle) is reachable and operable by keyboard, not a separate component to build.

admin/ follows the same rule

It is one more feature folder, not a second app — it sits beside menu/, cart/ and checkout/ and reuses the same ui/, hooks/ and api/ folders rather than duplicating them.